

Arrays - Simple Hinglish Summary

Source: <https://javascript.info/array>

Main baat

Objects **keyed collections** store karte hain. Lekin aksar hume **ordered collection** chahiye: 1st, 2nd, 3rd element, aise. Jaise users, goods, HTML elements ki list.

Iske liye object convenient nahi hai, kyunki wo elements ka order manage karne ke methods nahi deta. Existing properties ke **beech me** nayi property nahi daal sakte.

Ordered collections ke liye special data structure **Array** hai.

1. Declaration

Khali array banane ke do tarike:

```
javascript  
  
let arr = new Array();  
let arr = [];
```

Lagbhag hamesha dusra (`[]`) use hota hai. Shuru me elements bhi de sakte hain:

```
javascript  
  
let fruits = ["Apple", "Orange", "Plum"];
```

Elements **0** se number hote hain. Number se element milta hai:

```
javascript  
  
let fruits = ["Apple", "Orange", "Plum"];  
  
alert( fruits[0] ); // Apple  
alert( fruits[1] ); // Orange  
alert( fruits[2] ); // Plum
```

Element **badal** sakte hain:

```
javascript  
  
fruits[2] = 'Pear'; // ["Apple", "Orange", "Pear"]
```

Ya **naya add** kar sakte hain:

javascript

```
fruits[3] = 'Lemon'; // ["Apple", "Orange", "Pear", "Lemon"]
```

Elements ki total ginti (**length**) hai:

javascript

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits.length ); // 3
```

Poora array (**alert**) se bhi dikha sakte hain:

javascript

```
alert( fruits ); // Apple,Orange,Plum
```

Array me kisi bhi type ke elements ho sakte hain:

javascript

```
// mix of values
let arr = [ 'Apple', { name: 'John' }, true, function() { alert('hello'); } ];

// index 1 ka object lo aur uska name dikhao
alert( arr[1].name ); // John

// index 3 ka function lo aur chalo
arr[3](); // hello
```

Trailing comma: object ki tarah array ke end me bhi comma laga sakte hain. Isse items add/remove karna aasan hota hai kyunki saari lines ek jaisi hoti hain:

javascript

```
let fruits = [
  "Apple",
  "Orange",
  "Plum",
];
```

2. `at` se aakhri elements lena

Ye recent addition hai. Purane browsers me polyfill chahiye.

Kuch languages me aakhri element ke liye `fruits[-1]` chalta hai, lekin **JS me nahi chalta**. Result `undefined` hota hai, kyunki brackets ka index literally liya jata hai.

Aakhri element ka index khud calculate kar sakte hain: `fruits[fruits.length - 1]`:

javascript

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits[fruits.length-1] ); // Plum
```

Thoda bhaari hai kyunki variable ka naam do baar likhna padta hai. Chhota syntax:

`fruits.at(-1)`:

javascript

```
let fruits = ["Apple", "Orange", "Plum"];

// fruits[fruits.length-1] ke barabar
alert( fruits.at(-1) ); // Plum
```

`arr.at(i)`:

- `i >= 0` ho to `arr[i]` ke bilkul barabar
- `i` negative ho to **array ke end se peeche** ginta hai

3. `pop/push`, `shift/unshift` methods

Queue array ka sabse common use hai. Isme do operations hote hain:

- `push`: end me element jodta hai
- `shift`: shuru se element leta hai, aur queue aage badhti hai (dusra element pehla ban jata hai)

Stack ek aur data structure hai:

- `push`: end me element jodta hai
- `pop`: end se element leta hai

Stack me hamesha **end se** add/remove hota hai. Ise **cards ki gaddi** ki tarah samjho: naye cards upar rakhte hain aur upar se hi uthate hain.

Stack me jo **aakhri push hua wo pehle milta hai** (**LIFO**: Last-In-First-Out). Queue me **FIFO** (First-In-First-Out).

JS me arrays **dono (queue aur stack) ki tarah** kaam kar sakte hain. Shuru aur end dono se add/remove kar sakte hain. Is data structure ko **deque** kehte hain.

End ke saath kaam karne wale methods

pop: aakhri element nikalta hai aur return karta hai:

```
javascript

let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.pop() ); // "Pear" hata kar dikhaya

alert( fruits ); // Apple, Orange
```

`fruits.pop()` aur `fruits.at(-1)` dono aakhri element dete hain, lekin **pop** array me se **hata bhi deta hai**.

push: end me element jodta hai:

```
javascript

let fruits = ["Apple", "Orange"];

fruits.push("Pear");

alert( fruits ); // Apple, Orange, Pear
```

`fruits.push(...)` = `fruits[fruits.length] = ...`

Shuru ke saath kaam karne wale methods

shift: pehla element nikalta hai aur return karta hai:

```
javascript

let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.shift() ); // "Apple" hata kar dikhaya

alert( fruits ); // Orange, Pear
```

unshift: shuru me element jodta hai:

```
javascript
```

```
let fruits = ["Orange", "Pear"];

fruits.unshift('Apple');

alert( fruits ); // Apple, Orange, Pear
```

`push` aur `unshift` ek saath kai elements bhi jod sakte hain:

```
javascript

let fruits = ["Apple"];

fruits.push("Orange", "Peach");
fruits.unshift("Pineapple", "Lemon");

// ["Pineapple", "Lemon", "Apple", "Orange", "Peach"]
alert( fruits );
```

4. Internals (andar kya hota hai)

Array ek special type ka object hai. Square brackets `arr[0]` wahi object syntax hai, `obj[key]` jaisa, jahan `arr` object hai aur numbers keys hain.

Ye objects ko extend karke ordered data ke special methods aur `length` property deta hai. Lekin **andar se ye object hi hai.**

JS me sirf 8 basic data types hain. **Array object hai**, isliye object ki tarah behave karta hai. Jaise **reference se copy** hota hai:

```
javascript

let fruits = ["Banana"]

let arr = fruits; // reference se copy (do variables ek hi array ko point karte)

alert( arr === fruits ); // true

arr.push("Pear"); // reference se array badla

alert( fruits ); // Banana, Pear - ab 2 items
```

Lekin arrays ko khaas banata hai unki internal representation. Engine elements ko **memory ke contiguous (ek ke baad ek) area** me store karne ki koshish karta hai, aur aur optimizations bhi lagata hai jisse arrays bahut tez chalein.

Ye sab tab **toot jata hai** jab hum array ko "ordered collection" ki tarah nahi, balki **regular object ki tarah** use karte hain. Jaise technically ye possible hai:

```
javascript

let fruits = []; // array banaya

fruits[99999] = 5; // length se bahut bade index par property assign ki

fruits.age = 25; // kisi bhi naam ki property banayi
```

Ye possible hai kyunki arrays base me objects hain. Lekin engine dekhta hai ki hum array ko regular object ki tarah use kar rahe hain, aur **array-specific optimizations band ho jate hain**, unke fayde khatam.

Array ka galat use karne ke tarike:

- **Non-numeric property** add karna: `arr.test = 5`
- **Holes** banana: `arr[0]` phir `arr[1000]` (beech me kuch nahi)
- **Ulte order** me bharna: `arr[1000]`, `arr[999]`, aise

Arrays ko **ordered data** ke special structures ki tarah socho. Agar arbitrary keys chahiye to shayad tumhe **regular object** `{}` chahiye.

5. Performance

`push/pop` tez hain, `shift/unshift` dheere.

Array ke end ke saath kaam karna shuru se tez kyu hai? `fruits.shift()` me:

1. Index `0` wala element hatana
2. **Saare elements ko left me khisakana**, renumber karna (`1` se `0`, `2` se `1`, aise)
3. `length` update karna

Array me jitne zyada elements, utna zyada time, utne zyada memory operations.

`unshift` me bhi aisa hi: shuru me element jodne ke liye pehle existing elements ko right me khisakana padta hai.

`push/pop` me kuch khisakana nahi padta. `pop` sirf index saaf karta hai aur `length` chhota karta hai. **Baaki elements ke indexes wahi rehte hain, isliye `pop` bahut tez hai.** `push` ke saath bhi aisa hi hai.

6. Loops

Array par cycle karne ka sabse purana tarika indexes par `for` loop:

```
javascript
```

```
let arr = ["Apple", "Orange", "Pear"];

for (let i = 0; i < arr.length; i++) {
  alert( arr[i] );
}
```

Arrays ke liye ek aur loop `for..of`:

```
javascript

let fruits = ["Apple", "Orange", "Plum"];

// array elements par iterate karta hai
for (let fruit of fruits) {
  alert( fruit );
}
```

`for..of` current element ka **number nahi**, sirf **value** deta hai, lekin aksar wahi kaafi hota hai. Aur chhota bhi hai.

Technically arrays objects hain, isliye `for..in` bhi chal sakta hai:

```
javascript

let arr = ["Apple", "Orange", "Pear"];

for (let key in arr) {
  alert( arr[key] ); // Apple, Orange, Pear
}
```

Lekin ye bura idea hai. Problems:

1. `for..in` **saari properties** par chalta hai, sirf numeric par nahi. Browser me **"array-like" objects** hote hain jo array jaise dikhte hain (`length` aur indexes hote hain) lekin unme aur non-numeric properties aur methods bhi ho sakte hain, jo `for..in` list kar dega.
2. `for..in` generic objects ke liye optimize hai, arrays ke liye nahi, isliye **10-100 guna dheera** hai.

Aam taur par arrays ke liye `for..in` use mat karo.

7. `length` ke bare me

`length` array badalne par automatically update hoti hai. Sahi kahein to ye values ki ginti nahi, balki **sabse bada numeric index + 1** hai.

javascript

```
let fruits = [];  
fruits[123] = "Apple";  
  
alert( fruits.length ); // 124
```

Arrays ko aise aam taur par use nahi karte.

`length` **writable** bhi hai. Badhao to kuch khaas nahi hota. Ghatao to array truncate ho jata hai, aur ye wapas nahi hota:

javascript

```
let arr = [1, 2, 3, 4, 5];  
  
arr.length = 2; // 2 elements tak kaat diya  
alert( arr ); // [1, 2]  
  
arr.length = 5; // length wapas badhayi  
alert( arr[3] ); // undefined: values wapas nahi aati
```

Array khali karne ka sabse simple tarika: `arr.length = 0;`

8. `new Array()`

Array banane ka ek aur syntax:

javascript

```
let arr = new Array("Apple", "Pear", "etc");
```

Ye kam use hota hai kyunki `[]` chhota hai. Isme ek **tricky feature** bhi hai.

Agar `new Array` ko **sirf ek argument (number)** dete hain, to wo **bina items ke, lekin diye gaye length ka array** banata hai:

javascript

```
let arr = new Array(2); // kya [2] ka array banega?  
  
alert( arr[0] ); // undefined! koi element nahi.  
  
alert( arr.length ); // length 2
```

Aise surprises se bachne ke liye aam taur par square brackets use karte hain.

9. Multidimensional arrays

Arrays ke items bhi arrays ho sakte hain. Matrices store karne ke liye kaam aata hai:

javascript

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

alert( matrix[0][1] ); // 2, pehle inner array ki dusri value
```

10. toString

Arrays ka apna `toString` hota hai jo **comma-separated list** return karta hai:

javascript

```
let arr = [1, 2, 3];

alert( arr ); // 1,2,3
alert( String(arr) === '1,2,3' ); // true
```

Ye bhi try karo:

javascript

```
alert( [] + 1 ); // "1"
alert( [1] + 1 ); // "11"
alert( [1,2] + 1 ); // "1,21"
```

Arrays me `Symbol.toPrimitive` aur kaam ka `valueOf` nahi hota, sirf `toString` conversion hota hai. Isliye `[]` khali string, `[1]` "1", `[1,2]` "1,2" ban jata hai.

Binary plus jab string ke saath kuch jodta hai to use bhi string me badal deta hai:

javascript

```
alert( "" + 1 ); // "1"
alert( "1" + 1 ); // "11"
alert( "1,2" + 1 ); // "1,21"
```

11. Arrays ko (==) se compare mat karo

JS me arrays ko (==) se compare nahi karna chahiye. Is operator me arrays ke liye koi special treatment nahi hai, ye unhe kisi bhi object ki tarah handle karta hai.

Rules yaad karo:

- Do objects (==) tabhi barabar hain jab wo **ek hi object ke references** hon.
- (==) ka ek argument object aur dusra primitive ho to object primitive me convert hota hai.
- (null) aur (undefined) ek dusre ke barabar hain, aur kisi ke nahi.

Strict (===) aur simple hai: conversion nahi karta.

To arrays (==) se compare karo to wo **kabhi barabar nahi** hote, jab tak dono variables bilkul ek hi array ko refer na karte hon:

```
javascript
alert( [] == [] );    // false
alert( [0] == [0] );  // false
```

Ye technically alag objects hain. (==) **item-by-item comparison** nahi karta.

Primitives ke saath comparison ajeeb results de sakta hai:

```
javascript
alert( 0 == [] );    // true
alert('0' == [] );   // false
```

Dono cases me primitive ko array object se compare kar rahe hain. Array ([]) comparison ke liye primitive me convert hokar khali string (') ban jata hai. Phir comparison primitives ke saath aage badhta hai:

```
javascript
// [] ke '' banne ke baad
alert( 0 == '' );    // true ('' number 0 me convert hoti hai)

alert('0' == '' );   // false (conversion nahi, alag strings)
```

To arrays kaise compare karein? Simple: (==) use mat karo. **Loop me item-by-item compare karo** (ya agle chapter ke iteration methods se).

Summary

Array ek **special type ka object** hai jo **ordered data items** store aur manage karne ke liye bana hai.

Declaration:

```
javascript

// square brackets (aam)
let arr = [item1, item2...];

// new Array (bahut kam)
let arr = new Array(item1, item2...);
```

`new Array(number)` diye gaye length ka array banata hai, lekin **bina elements** ke.

- `length` array ki lambai hai, ya sahi kahein to aakhri numeric index + 1. Array methods se ye automatic adjust hoti hai.
- `length` haath se chhoti karo to array **truncate** ho jata hai.

Elements lena:

- Index se: `arr[0]`
- `at(i)` method: negative indexes allow karta hai. `i` negative ho to end se peeche ginta hai. `i >= 0` ho to `arr[i]` jaisa.

Array ko deque ki tarah use karne ke operations:

- `push(...items)`: end me jodta hai
- `pop()`: end se hata kar return karta hai
- `shift()`: shuru se hata kar return karta hai
- `unshift(...items)`: shuru me jodta hai

Elements par loop:

- `for (let i=0; i<arr.length; i++)`: sabse tez, purane browsers ke saath compatible
- `for (let item of arr)`: sirf items ke liye modern syntax
- `for (let i in arr)`: **kabhi mat use karo**

Arrays compare karne ke liye `==` (aur `>`, `<`, etc.) **mat use karo**, kyunki inme arrays ke liye special treatment nahi hai, wo unhe aam objects ki tarah handle karte hain, jo aam taur par hum nahi chahte. Iski jagah `for...of` se item-by-item compare karo.

Agle chapter me **Array methods** hain.

Practice Tasks (Answers)

1. Kya array copy hua?

javascript

```
let fruits = ["Apples", "Pear", "Orange"];

// "copy" me nayi value push ki
let shoppingCart = fruits;
shoppingCart.push("Banana");

// fruits me kya hai?
alert( fruits.length ); // ?
```

Jawab: ④. Arrays objects hain, isliye `shoppingCart` aur `fruits` dono **ek hi array ke references** hain.

2. Array operations:

1. `styles` array banao: "Jazz", "Blues"
2. End me "Rock-n-Roll" jodo
3. Beech wali value "Classics" se badlo (kisi bhi odd length array ke liye kaam kare)
4. Pehli value hata kar dikhao
5. Shuru me `Rap` aur `Reggae` jodo

javascript

```
let styles = ["Jazz", "Blues"];
styles.push("Rock-n-Roll");
styles[Math.floor((styles.length - 1) / 2)] = "Classics";
alert( styles.shift() );
styles.unshift("Rap", "Reggae");
```

3. Array context me call:

javascript

```
let arr = ["a", "b"];

arr.push(function() {
  alert( this );
});

arr[2](); // ?
```

Jawab: `a,b,function(){...}`. `arr[2]()` syntactically purana `obj[method]()` hai, jahan `obj` = `arr`. Isliye function ko `this = arr` milta hai aur wo poora array dikhata hai (ab isme 3 values hain).

4. Input numbers ka sum (`sumInput`):

```
javascript

function sumInput() {

  let numbers = [];

  while (true) {

    let value = prompt("A number please?", 0);

    // cancel karna hai?
    if (value === "" || value === null || !isFinite(value)) break;

    numbers.push(+value);
  }

  let sum = 0;
  for (let number of numbers) {
    sum += number;
  }
  return sum;
}

alert( sumInput() );
```

`prompt` ke turant baad `value` ko number me nahi badalte, kyunki tab khali string (rukne ka signal) aur zero (valid number) me farak nahi kar paate. Isliye conversion baad me karte hain.

5. Maximal subarray (getMaxSubSum): Input array of numbers, jaise `[1, -2, 3, 4, -9, 6]`.

Kaam: **contiguous subarray** jiska sum sabse bada ho, wo sum return karo. Saare negative hon to koi element nahi lete, sum `0`.

Dheera solution $O(n^2)$: har element se shuru hone wale saare subarrays ke sums nikalo:

javascript

```
function getMaxSubSum(arr) {  
  let maxSum = 0; // koi element nahi liya to 0  
  
  for (let i = 0; i < arr.length; i++) {  
    let sumFixedStart = 0;  
    for (let j = i; j < arr.length; j++) {  
      sumFixedStart += arr[j];  
      maxSum = Math.max(maxSum, sumFixedStart);  
    }  
  }  
  
  return maxSum;  
}
```

Array size 2 guna karo to algorithm 4 guna dheera chalta hai.

Tez solution $O(n)$: array par chalte hue current partial sum `s` rakho. Kabhi `s` negative ho jaye to `s = 0` kar do. Aise saare `s` me se maximum jawab hai:

javascript

```
function getMaxSubSum(arr) {
  let maxSum = 0;
  let partialSum = 0;

  for (let item of arr) {
    partialSum += item;           // partialSum me jodo
    maxSum = Math.max(maxSum, partialSum); // maximum yaad rakho
    if (partialSum < 0) partialSum = 0; // negative ho to zero
  }

  return maxSum;
}

alert( getMaxSubSum([-1, 2, 3, -9]) ); // 5
alert( getMaxSubSum([-1, 2, 3, -9, 11]) ); // 11
alert( getMaxSubSum([-2, -1, 1, 2]) ); // 3
alert( getMaxSubSum([100, -9, 2, -3, 5]) ); // 100
alert( getMaxSubSum([1, 2, 3]) ); // 6
alert( getMaxSubSum([-1, -2, -3]) ); // 0
```

Ye array par **sirf ek pass** leta hai, isliye time complexity **O(n)** hai.

Quick Summary

Kaam	Kaise
Array banana	<code>let arr = [1, 2, 3];</code>
Element lena	<code>arr[0]</code> , aakhri: <code>arr.at(-1)</code>
Lambai	<code>arr.length</code>
End me jodna / hatana	<code>push(x)</code> / <code>pop()</code>
Shuru me jodna / hatana	<code>unshift(x)</code> / <code>shift()</code>
Speed	<code>push/pop</code> tez , <code>shift/unshift</code> dheere
Loop	<code>for (let x of arr)</code> ((<code>for..in</code>) kabhi nahi)
Khali karna	<code>arr.length = 0;</code>
<code>new Array(2)</code>	2 length ka khali array ((<code>[2]</code>) nahi!)
Multidimensional	<code>matrix[0][1]</code>
<code>[] == []</code>	<code>false</code> (alag objects)
Compare karna	Loop me item-by-item
Copy	Reference se hoti hai ((<code>let b = a;</code>) alag copy nahi)
Stack	<code>push</code> + <code>pop</code> (LIFO)
Queue	<code>push</code> + <code>shift</code> (FIFO)